

Lab 1: Retraining MicroGPT

New Data, New Distribution, Same Machine

ECON 470 — Microeconomics of Artificial Intelligence

Spring 2026

1 Overview

In lecture you watched a 200-line GPT learn to generate baby names. In this lab you retrain the same model on a completely different dataset — **dinosaur genus names** — and observe what changes and what doesn't. Along the way you will interpret the loss function, connect architectural limits to the data, and see firsthand that the model is a pure prediction machine: change the data, and the predictions change with it.

Groups: 2–3 students. At least one member of each group must have experience running Python code (installing packages, editing a script, running it from the terminal or a notebook). The other members do not need to know how to code — the questions are about *interpreting* what the model does, not about programming. **Only one submission per group, please.**

Submit: A short writeup with your answers to the numbered questions below. Include screenshots of your terminal output where indicated.

Time estimate: 2–3 hours, including training runs. You can work on written answers while the model trains. Start early — do not leave this for the night before.

Points: 100 total. See point values next to each question.

If you get stuck: The MicroGPT Tutor can help you work through problems without giving away the answer: <https://chatgpt.com/g/g-69cbe9820bb481918c188ef086d7a2d8-microgpt-tutor>. Use it sparingly — you'll learn more by wrestling with the questions yourself first.

2 Setup

Download the following files from Canvas:

- `microgpt.py` — the same 200-line GPT from lecture
- `dinosaurs.txt` — 1,835 dinosaur genus names, one per line

Place both files in the same folder. Open `microgpt.py` in a text editor and make one change: on **line 15**, replace

```
if not os.path.exists('input.txt'):
    import urllib.request
    names_url = 'https://raw.githubusercontent.com/karpathy/makemore/988aa59/names.txt'
    urllib.request.urlretrieve(names_url, 'input.txt')
docs = [line.strip() for line in open('input.txt') if line.strip()]
```

with

```
docs = [line.strip() for line in open('dinosaurs.txt') if line.strip()]
```

Save the file and run it:

```
python3 microgpt.py
```

Training takes a few minutes. When it finishes, the model will print 20 generated dinosaur names.

3 Problem 1: Training on Dinosaurs

3.1 Part A: The generated names

Look at the 20 sample names the model produces.

- (1) Copy the 20 generated names into your writeup. For each name, briefly note whether it *sounds like* a plausible dinosaur name. How many of the 20 would you say are convincing? What patterns did the model pick up (common endings, typical syllable structure, etc.)? [5 pts]

3.2 Part B: Interpreting the loss

The model prints a loss value at each training step. You should see the loss start high and decrease during training.

- (2) **Record the initial loss** (step 1) and the **final loss** (last step). Convert each to a probability using the formula from Lecture 1:

$$P = e^{-\text{loss}}$$

Interpret each probability in plain language. What does the initial probability tell you about the model before training? What does the final probability tell you about the model after training? [5 pts]

- (3) **Compare to the baby names model.** In the baby names exercise, the initial loss was approximately 3.3 and the final loss was approximately 2.37. Are your dinosaur losses higher or lower? Why might this be the case? (*Hint*: think about how many names are in each dataset and how varied the character patterns are.) [5 pts]

3.3 Part C: The context window problem

The default `block_size` in `microgpt.py` is **16**, meaning the model can only “see” the last 16 characters when predicting the next one. The longest baby name in the original dataset is 15 characters, so this was fine.

- (4) How long is the longest dinosaur name in the dataset? How many dinosaur names are longer than 16 characters? Run the following code to find out:

```
names = [line.strip() for line in open('dinosaurs.txt')]
print("Longest:", max(names, key=len), f"({len(max(names, key=len))} chars)")
print("Longer than 16:", sum(len(n) > 16 for n in names))
```

What does this mean for the model’s ability to learn those names? *[5 pts]*

3.4 Part D: Expanding the context window

Now change `block_size`. On **line 77** of `microgpt.py`, change

```
block_size = 16
```

to

```
block_size = 24
```

Save and re-run the model from scratch (`python3 microgpt.py`).

- (5) Copy the 20 new sample names. Do they look different from the `block_size = 16` names? Are they more or less plausible? Do you notice any difference in the length of the generated names? *[5 pts]*
- (6) Record the initial and final loss values for this run. Convert them to probabilities as before. Did the loss improve, get worse, or stay about the same compared to the `block_size = 16` run? What does this tell you about the relationship between architectural capacity and prediction quality? *[5 pts]*

4 Problem 2: Backpropagation and the Value Class

In lecture we saw a small example of automatic differentiation: $L = (a \times b) + a$ with $a = 2$, $b = 3$. The `Value` class in `microgpt.py` (lines 30–72) made this possible. In this problem you’ll make sure you understand what it does and why the model needs it.

4.1 Part A: What the Value class does

For this part, the section **Tracking every calculation** from this interactive resource may be valuable: <https://growingswe.com/blog/microgpt>

- (7) In your own words (2–3 sentences), explain what the `Value` class does. A `Value` object stores a scalar (the `.data` attribute), but it also stores *children* and *local gradients*. Why does it need those? [5 pts]
- (8) When you call `L.backward()`, what happens? Describe in plain language what the `backward` method computes and how it moves through the computation graph. [5 pts]

4.2 Part B: Marginal products of a CES production function

In intermediate microeconomics you worked with the CES (Constant Elasticity of Substitution) production function:

$$Y = (\alpha \cdot L^\rho + (1 - \alpha) \cdot K^\rho)^{1/\rho}$$

where L is labor, K is capital, α is the input share parameter, and ρ controls the elasticity of substitution.

Use the following parameter values: $\alpha = 0.5$, $\rho = 0.5$, $L = 4$, $K = 9$.

- (9) **By hand**, compute the output Y . Show your work. [3 pts]
- (10) **By hand**, compute the marginal product of labor $\frac{\partial Y}{\partial L}$ and the marginal product of capital $\frac{\partial Y}{\partial K}$. Show your work using the chain rule. [10 pts]
- (11) **Now verify with the Value class.** Copy the `Value` class from the Lecture 1 slides into a new Python file (or into a Python console) and run:

```
L = Value(4.0)
K = Value(9.0)
Y = (0.5 * L**0.5 + 0.5 * K**0.5) ** 2
Y.backward()
print(f"Y = {Y.data}")
print(f"MP_L = {L.grad}")
print(f"MP_K = {K.grad}")
```

Do the numbers match your hand calculations? [3 pts]

4.3 Part C: Why MicroGPT needs this

- (12) In `microgpt.py`, every model parameter (every weight in the embedding matrices, the attention matrices, etc.) is a `Value` object, not an ordinary number. The loss function is also a `Value`. In 2–3 sentences, explain *why*. What does calling `loss.backward()` give the training loop, and how does the training loop use it to make the model's predictions better? [10 pts]

5 Problem 3: MicroOLS — Regression by Gradient Descent

In your econometrics and statistics courses you estimated regressions using Ordinary Least Squares (OLS), which has a closed-form solution. In this problem you will estimate the exact same regression using *gradient descent and backpropagation* — the same optimization machinery that trains GPT. The point is to see that the substrate is identical: the `Value` class, the loss function, and the optimizer don't care whether they're fitting a line or training a language model.

Download `microols.py` and `oj.csv` from Canvas. Place them in the same folder and run:

```
python3 microols.py
```

The script estimates a log-linear demand curve for orange juice using 28,947 weekly supermarket scanner observations from three brands (Dominick's, Minute Maid, and Tropicana):

$$\ln(Q) = a \cdot \ln(P) + b_{\text{mm}} \cdot \text{MinuteMaid} + b_{\text{tr}} \cdot \text{Tropicana} + b_0$$

where Q is weekly unit sales, P is price, and the brand variables are dummies (Dominick is the store brand and is the omitted category). The slope a is the **price elasticity of demand**.

5.1 Part A: Read the code

Open `microols.py` and compare it side-by-side with `microgpt.py`.

- (13) List three code blocks that are the **same** between `microols.py` and `microgpt.py`. [3 pts]
- (14) List three things that are **different**. [3 pts]

5.2 Part B: Training output

- (15) Record the final values of the four parameters: **a** (the elasticity), **b_mm** (Minute Maid dummy), **b_tr** (Tropicana dummy), and **b0** (the intercept). Is demand elastic or inelastic? What does the sign of **a** tell you? What do the brand coefficients tell you about how sales differ across brands, holding price constant? [5 pts]
- (16) **Compare to the closed-form OLS solution.** If you run OLS on this data using the standard formula (in Excel, R, Python, or any software you like), you get $a = -3.1387$, $b_{\text{mm}} = 0.8702$, $b_{\text{tr}} = 1.5299$, and $b_0 = 10.8288$. How close did gradient descent get? What does it tell you that the same algorithm that trains GPT recovers the same answer as the closed-form formula you learned in econometrics? [5 pts]

5.3 Part C: Inference — sampling from the conditional distribution

After training, the script samples 20 draws from the conditional distribution of $Q \mid P = \$3.00$, brand = Tropicana. Each draw computes $\hat{y} = a \cdot \ln(3.00) + b_{\text{tr}} + b_0$, adds random noise from $N(0, s_e^2)$ (where s_e is the residual standard error), and converts back to units via $Q = e^{\ln(Q)}$.

- (17) Look at the 20 sampled values of Q . What is the range (smallest to largest)? Does the variation surprise you? [5 pts]
- (18) In `microgpt.py`, inference samples from a probability distribution over the next character. In `microols.py`, inference samples from a probability distribution over sales given price and brand. In one or two sentences, explain what is analogous and what is different about these two forms of “prediction.” [3 pts]

5.4 Part D: Connecting to the course

- (19) **Synthesis.** You have now seen the same optimization engine — `Value + backward()` + Adam — used to train a language model (`microgpt`) and to estimate a demand curve (`microols`). In a short paragraph, reflect on what this tells you about the claim from Gans Chapter 1 that AI is, economically, a *prediction technology*. What does “prediction” mean in each case? What is the role of *judgment* in each case — who decides what to predict, what loss function to minimize, and what to do with the prediction? [10 pts]